

**République Tunisienne**
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
École Supérieure Privée d'Ingénierie et de Technologie — TEK-UP

# Conception et développement d'une plateforme d'inventaire logistique multi-canal

Rapport de Projet de Fin d'Études — Lot 2

Chapitre 3 — Conception de la solution

Réalisé par : **Harrabi Yosri**

Spécialité : Génie Logiciel et Systèmes d'Information

Encadrant professionnel : M. Ferchichi Marwen — VECTORYS Tunisie

Encadrante académique : Mme Zrelli Siwar — TEK-UP

Année universitaire 2025 — 2026

# Table des matières — Lot 2

Chapitre 3 — Conception de la solution

3.1 Architecture globale de la solution

3.2 Modèle de données et diagramme de classes global

3.3 Conception par fonctionnalité

3.3.1 Authentification

3.3.2 Création et configuration d'inventaire

3.3.3 Import du stock théorique

3.3.4 Comptage terrain

3.3.5 Gestion des écarts

3.3.6 Pilotage et suivi

3.4 Vue synthétique de l'API

Conclusion du chapitre

# Chapitre 3 — Conception de la solution

*Ce chapitre traduit les besoins formalisés au chapitre 2 en une conception technique exploitable. Il s'appuie sur le diagramme de cas d'utilisation global (Figure 2.4) et détaille, pour chaque domaine fonctionnel, un cas d'utilisation raffiné, les classes concernées et/ou un diagramme de séquence. Les captures d'écran sont prévues sous forme d'emplacements réservés, à compléter ultérieurement.*

## Introduction

La conception vise trois objectifs : (1) garantir la cohérence du modèle métier entre les canaux web et mobile, (2) isoler la logique métier dans l'API selon une Clean Architecture (CQRS/MediatR), et (3) préparer une réalisation progressive alignée sur les sprints 0 à 9, tous menés à terme.

Le fil conducteur reprend la structure du use case global : authentification, gestion des inventaires, comptage terrain, gestion des écarts, puis pilotage. Chaque sous-section suit le même schéma : rappel du cas global, spécification détaillée, diagramme(s) UML, justification des choix, emplacement pour illustration.

## 3.1 Architecture globale de la solution

La plateforme Vectorys Inventaire repose sur une architecture multi-canal three-tier. La couche présentation regroupe l'application web Angular (superviseur / administrateur) et l'application mobile Flutter (agent). La couche métier est exposée par une API REST ASP.NET Core 9, structurée en Domain, Application (commands/queries MediatR, FluentValidation) et Infrastructure (EF Core, SQL Server, ClosedXML, SignalR). La figure 3.1 rappelle cette organisation.

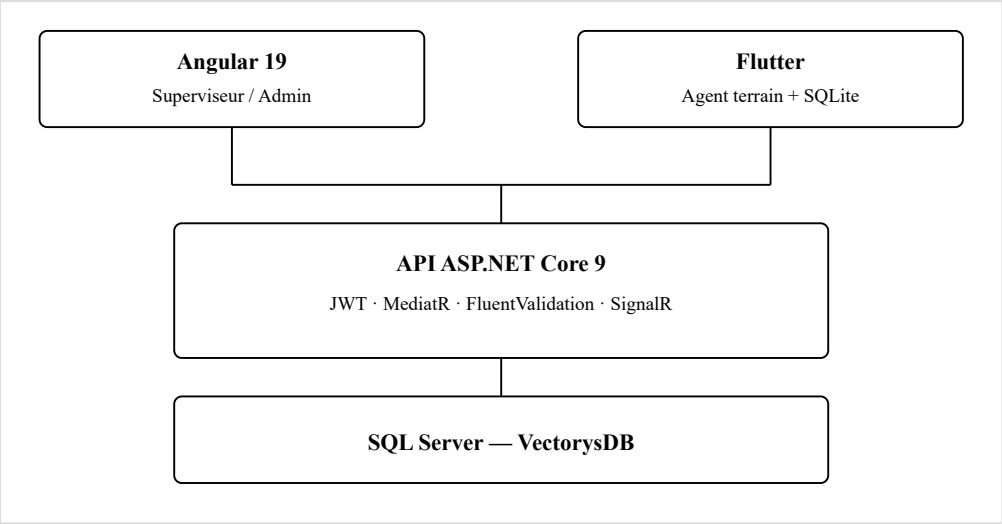



*Figure 3.1 — Architecture logique three-tier de Vectorys Inventaire*

Ce choix assure une source de vérité unique côté serveur, une séparation nette des responsabilités et une capacité d'évolution (nouveaux KPI, nouveaux passages) sans casser les clients. Le mode hors ligne mobile s'appuie sur SQLite local et une synchronisation différée, sans dupliquer la logique métier des quotas côté serveur.

## 3.2 Modèle de données et diagramme de classes global

Le modèle métier centralise sept entités principales. La chaîne structurante est : **Inventaire → Passage (C1/C2) → Emplacement → StockThéorique / Scan**. L'entité **Écart** relie inventaire, passage et emplacement après détection. Les indicateurs KPI ne constituent pas une entité persistée : ils sont calculés par agrégation.

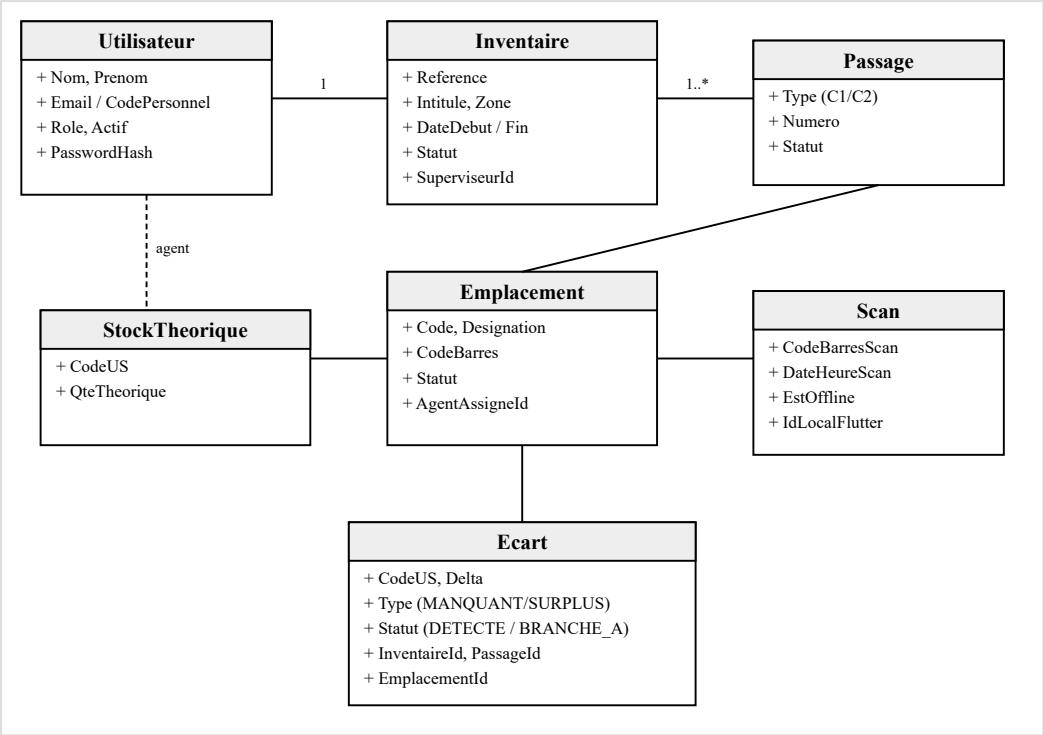



*Figure 3.2 — Diagramme de classes global (vue métier)*

| Énumération | Valeurs |
|---|---|
| RoleEnum | SUPERVISEUR, AGENT, ADMIN |
| StatutInventaireEnum | EN_PREPARATION, EN_COURS, CLOTURE |
| TypePassageEnum | C1, C2 |
| StatutEmplacementEnum | EN_ATTENTE, EN_COURS, TERMINE |
| TypeEcartEnum | MANQUANT, SURPLUS |
| StatutEcartEnum | DETECTE, TRAITE_BRANCHE_A |

## 3.3 Conception par fonctionnalité

Les sous-sections suivantes raffinent le use case global. Pour chaque fonctionnalité : fiche de cas d'utilisation, diagramme de séquence et/ou de classes ciblé, puis emplacement de capture.

### 3.3.1 Authentification

**Cas global de référence :** « S'authentifier ». Ce point d'entrée est commun au superviseur, à l'administrateur et à l'agent. Il conditionne l'accès aux autres domaines via un jeton JWT et des policies de rôle.

**Cas d'utilisation détaillé — S'authentifier**

| Élément | Description |
|---|---|
| Acteurs | Superviseur, Administrateur, Agent |
| Préconditions | Compte existant et actif dans la base |
| Scénario nominal | 1. L'acteur ouvre l'écran de connexion (web ou mobile).<br>2. Superviseur/Admin saisit email et mot de passe ; Agent saisit son code personnel.<br>3. L'API valide les identifiants (BCrypt) et le rôle.<br>4. Un JWT est émis (24 h superviseur/admin, 8 h agent).<br>5. Le client stocke le jeton et accède à son espace. |
| Alternatives | Identifiants invalides ; compte inactif ; mélange des modes de login → rejet 400/401 |
| Postconditions | Session authentifiée ; appels API autorisés selon le rôle |

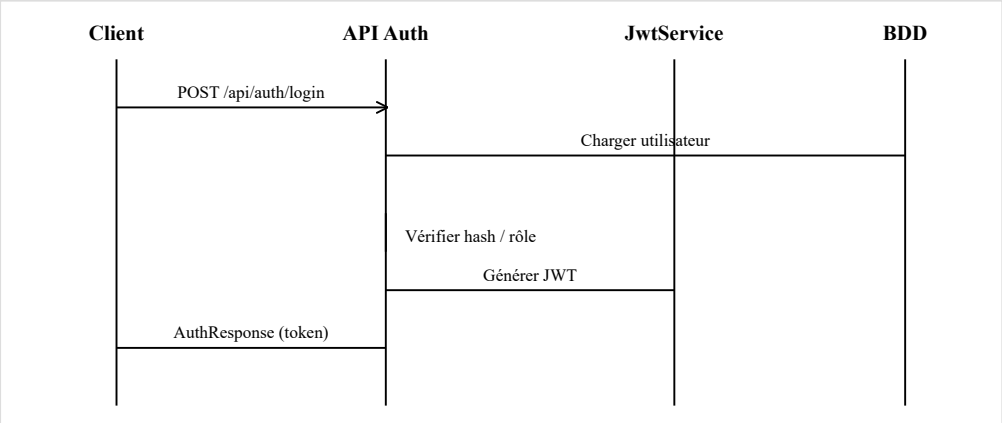



*Figure 3.3 — Diagramme de séquence : authentification*

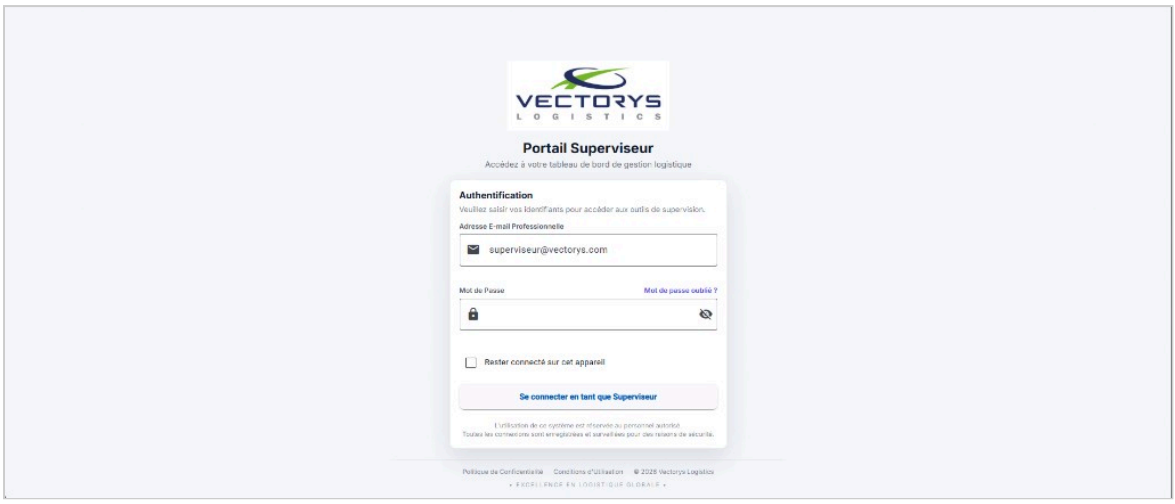



*Figure 3.4 — Interface d'authentification Angular (portail superviseur)*

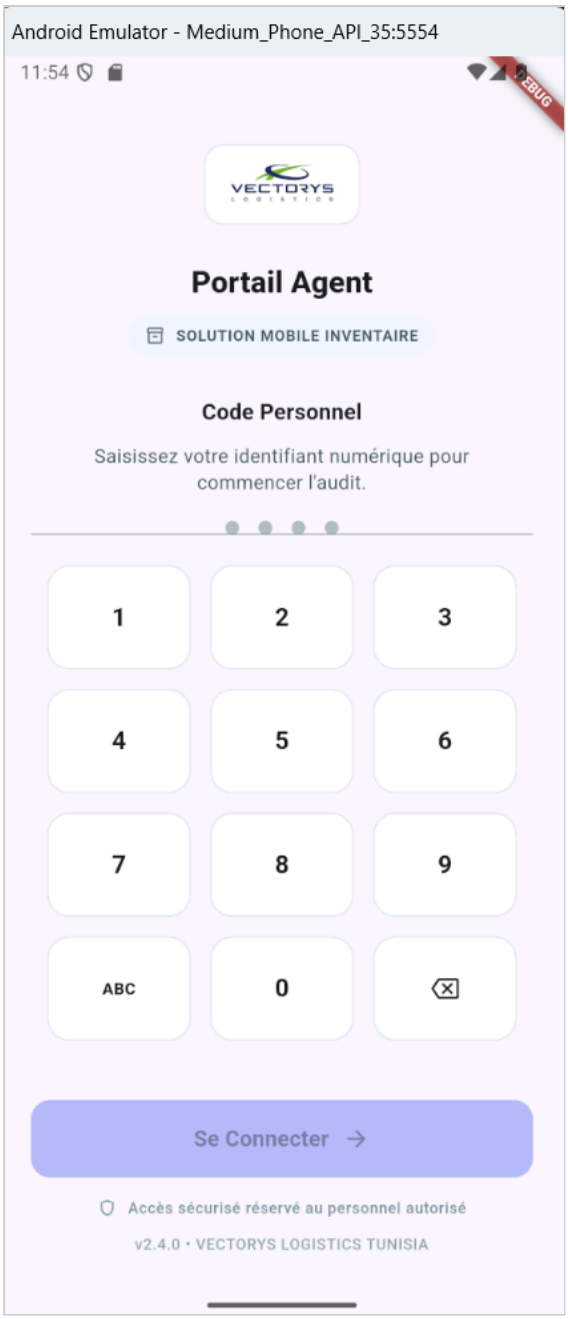



*Figure 3.5 — Interface d'authentification Flutter (portail agent)*

### 3.3.2 Création et configuration d'inventaire

**Cas global :** « Créer et configurer un inventaire ». Ce cas regroupe la création de la mission, l'affectation des agents et la génération automatique du passage C1.

**Cas d'utilisation détaillé — Créer et configurer un inventaire**

| Élément | Description |
|---|---|
| Acteur | Superviseur (authentifié) |
| Préconditions | JWT valide ; rôle SUPERVISEUR ou ADMIN |
| Scénario nominal | 1. Le superviseur saisit intitulé, zone et dates.<br>2. L'API génère la référence INV-{année}-{seq}.<br>3. Un passage C1 est créé automatiquement.<br>4. L'inventaire est enregistré en statut EN_PREPARATION. |
| Postconditions | Inventaire et passage C1 persistés ; prêts pour l'import Excel |

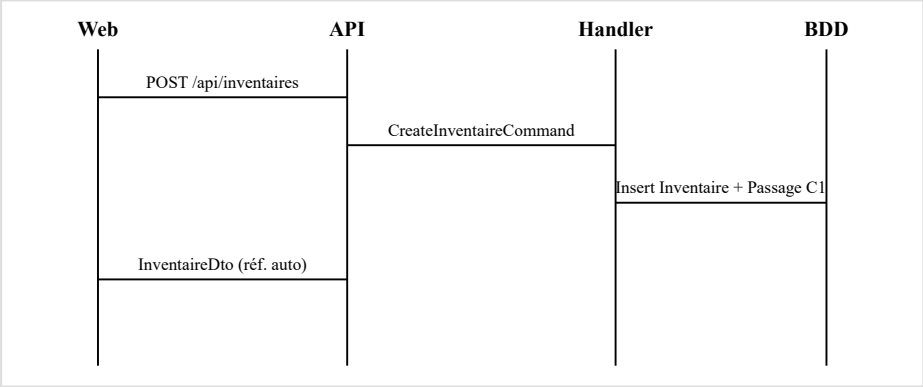



*Figure 3.6 — Diagramme de séquence : création d'inventaire et génération C1*

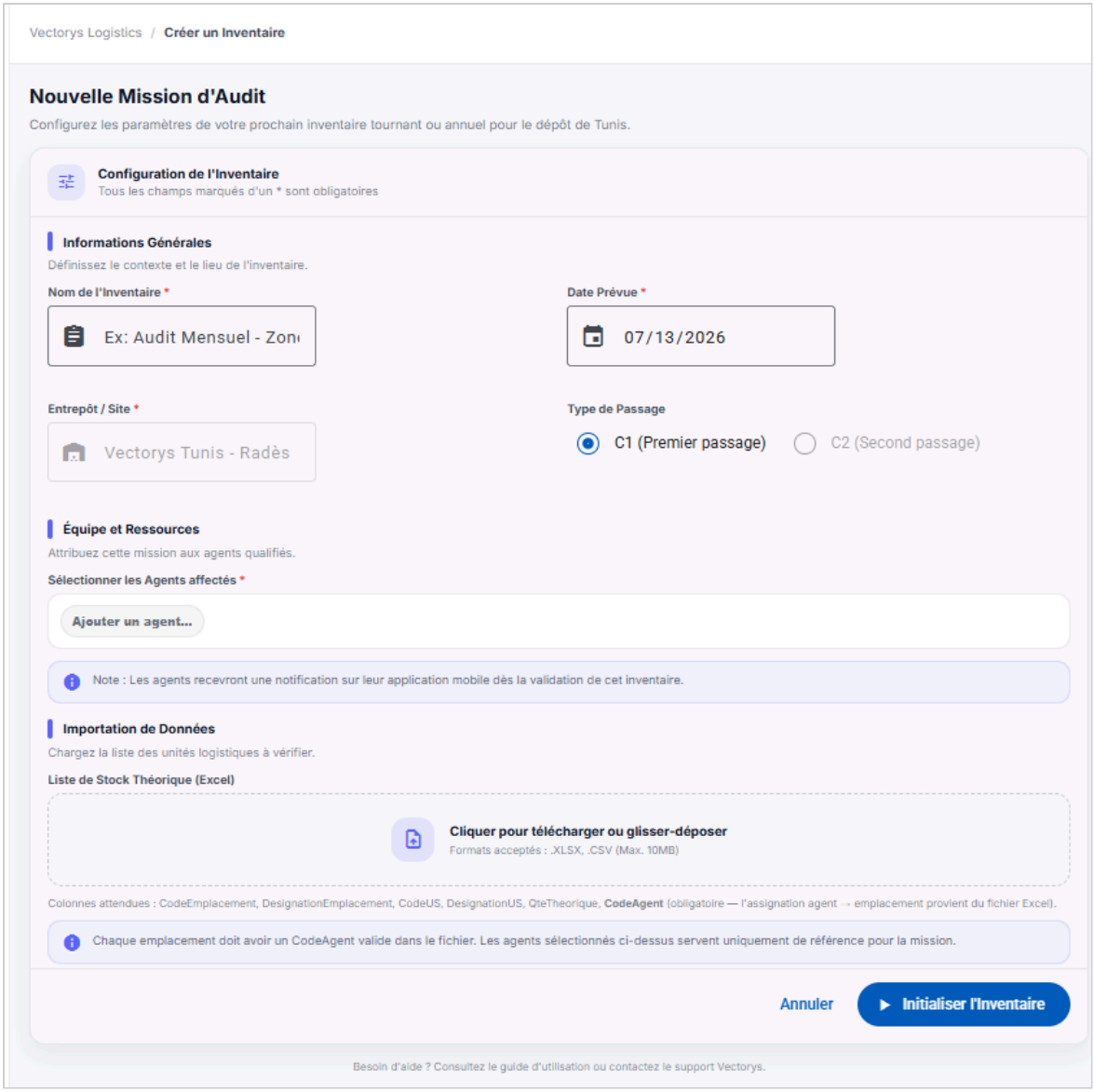



*Figure 3.7 — Formulaire de création / configuration d'inventaire*

### 3.3.3 Import du stock théorique

**Cas global :** « Importer le stock depuis un fichier Excel » (acteur complémentaire : Système ERP). L'import matérialise les emplacements, le stock théorique et l'assignation stricte via la colonne *CodeAgent*.

### Cas d'utilisation détaillé — Importer le stock

| Élément | Description |
|---|---|
| Acteurs | Superviseur ; Système ERP (fichier Excel) |
| Préconditions | Inventaire et passage C1 existants ; fichier .xlsx conforme |
| Scénario nominal | 1. Le superviseur téléverse le fichier sur le passage.<br>2. ClosedXML lit les lignes (CodeEmplacement, CodeUS, Qte, CodeAgent…).<br>3. Validation : CodeAgent obligatoire et cohérent.<br>4. Création batch des emplacements et stocks théoriques.<br>5. Inventaire passé EN_COURS ; rapport d'import renvoyé. |
| Alternatives | Colonnes manquantes, agent inconnu, fichier trop volumineux → rejet avec message |

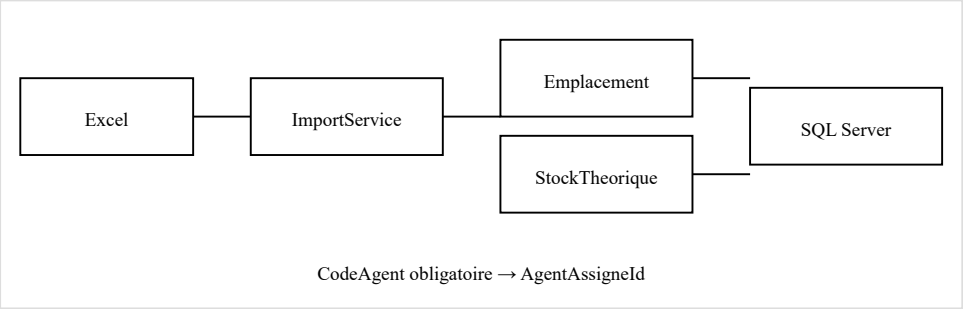



*Figure 3.8 — Vue conceptuelle de l'import Excel vers le modèle de données*

**[Insérer ici la capture d'écran de la zone d'import Excel / rapport d'assignation]**

Figure 3.9 — Import du stock théorique et validation CodeAgent

### 3.3.4 Comptage terrain

**Cas globaux :** « Consulter les emplacements assignés », « Réaliser le comptage des unités de stock », « Travailler en mode hors ligne » (extension). Ces cas couvrent le cœur métier agent : liste des tâches, boucle de scan, quotas, clôture automatique et synchronisation batch.

**Cas d'utilisation détaillé — Réaliser le comptage**

| Élément | Description |
|---|---|
| Acteur | Agent |
| Préconditions | Inventaire EN_COURS ; emplacements assignés à l'agent |
| Scénario nominal | 1. L'agent consulte ses emplacements (mes-emplacements).<br>2. Il scanne le code US ; le serveur vérifie le quota QteThéorique.<br>3. Le scan est enregistré ; la progression US est mise à jour.<br>4. Lorsque tous les US sont couverts, l'emplacement passe TERMINE automatiquement.<br>5. SignalR notifie le superviseur (NouveauScan / FinComptage). |
| Extension offline | Scans stockés en SQLite avec IdLocalFlutter ; sync POST /api/scans/batch au retour réseau |

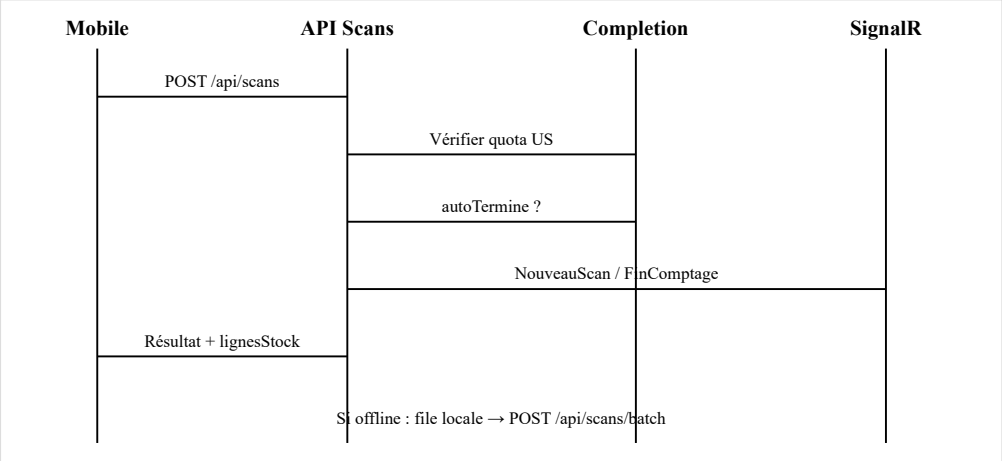



*Figure 3.10 — Diagramme de séquence : scan, quota et notification*

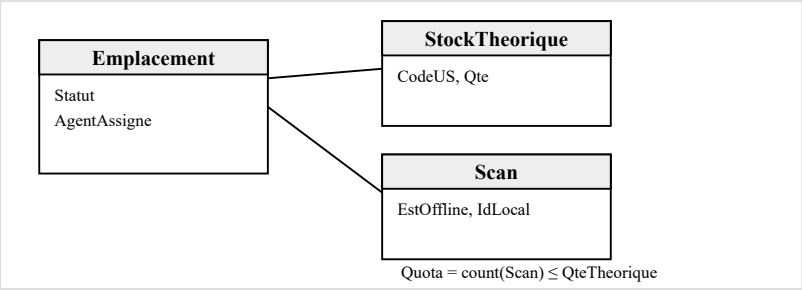



*Figure 3.11 — Classes concernées par le comptage terrain*




*Figure 3.12 — Emplacements assignés (mobile)*

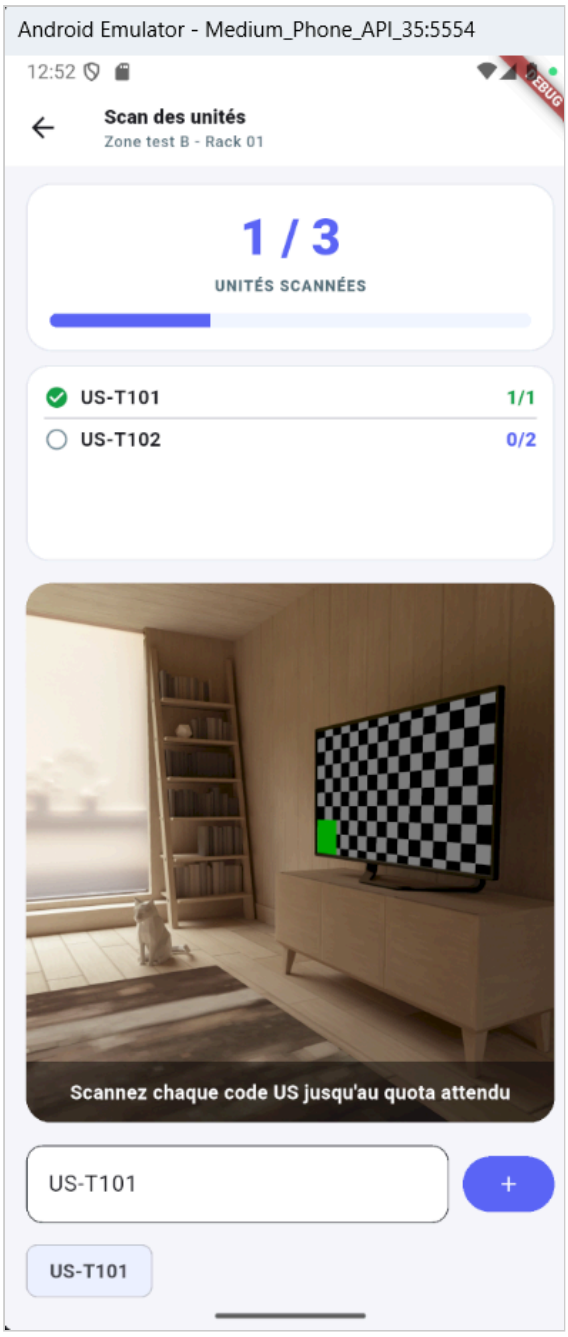



*Figure 3.13 — Scan en boucle et progression par unité de stock*

### 3.3.5 Gestion des écarts

**Cas globaux :** « Gérer les écarts et choisir une stratégie », extensions Branche A / Branche B, et « Générer le fichier d'inventaire final ». Après le C1, le superviseur compare stock théorique et quantités scannées, puis oriente le traitement.

**Cas d'utilisation détaillé — Gérer les écarts**

| Élément | Description |
|---|---|
| Acteur | Superviseur |
| Préconditions | Emplacements C1 en statut TERMINE ; détection lancée |
| Scénario nominal | 1. Détection des écarts MANQUANT / SURPLUS.<br>2. Consultation de la liste et des agrégats.<br>3a. *Branche A* : marquage TRAITE_BRANCHE_A, création C2, clonage des emplacements concernés.<br>3b. *Branche B* : recomptage global (extension).<br>4. Génération / export du fichier final d'inventaire. |

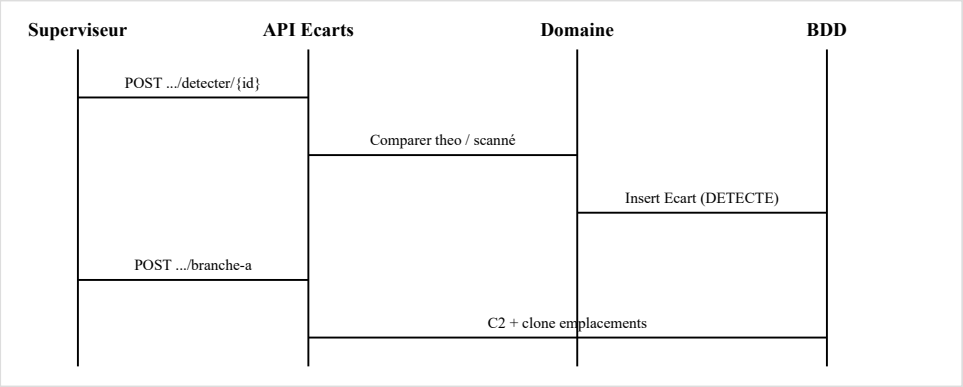



*Figure 3.14 — Diagramme de séquence : détection et Branche A*

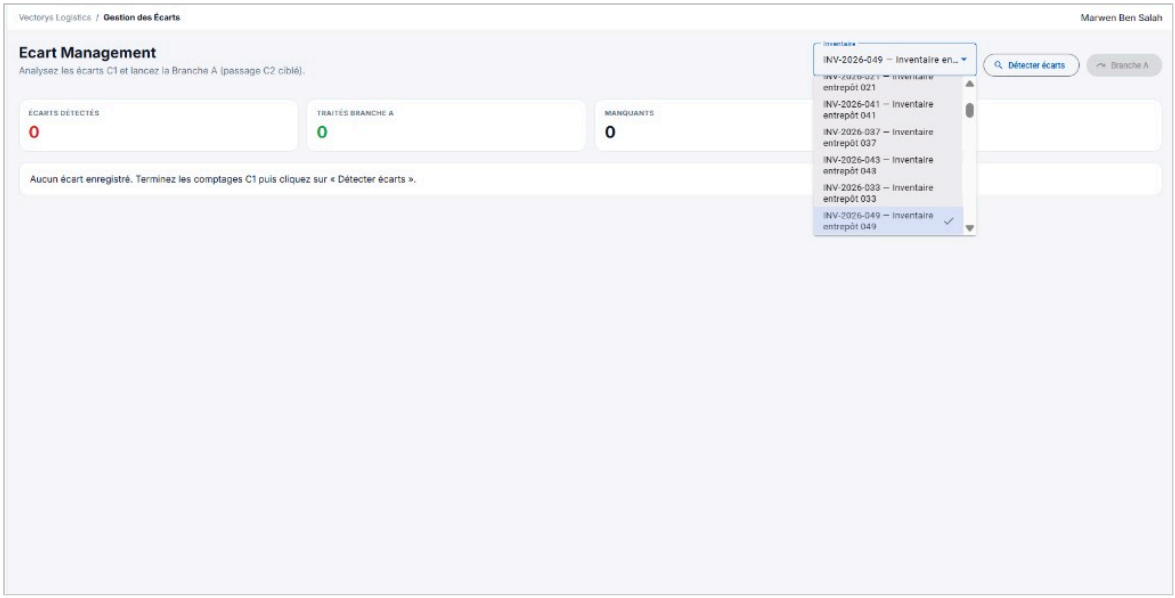



*Figure 3.15 — Liste des écarts et choix de stratégie (Branche A / B)*

### 3.3.6 Pilotage et suivi

**Cas globaux :** « Suivre l'avancement en temps réel » et « Consulter le tableau de bord KPI ». Le suivi s'appuie sur SignalR et l'endpoint de suivi ; les KPI sont des agrégats (précision, performance, qualité, clients, WMS).

**Cas d'utilisation détaillé — Consulter le tableau de bord KPI**

| Élément | Description |
|---|---|
| Acteur | Superviseur |
| Préconditions | Données d'inventaire / scans / écarts disponibles |
| Scénario nominal | 1. Le superviseur ouvre le module KPI.<br>2. Il applique éventuellement des filtres (inventaire, période, zone).<br>3. L'API agrège les indicateurs et renvoie le dashboard.<br>4. Les graphiques Chart.js affichent précision, performance, qualité, WMS. |

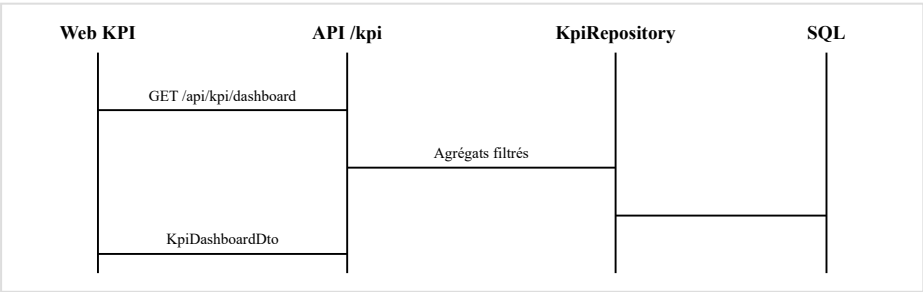



*Figure 3.16 — Diagramme de séquence : consultation KPI*

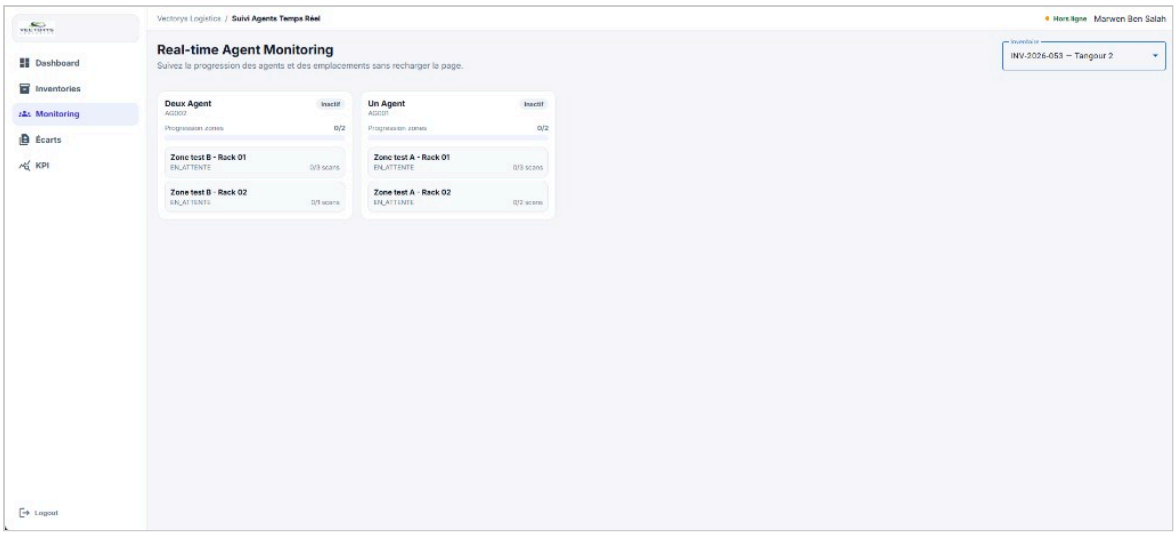



*Figure 3.17 — Suivi d'avancement en temps réel*

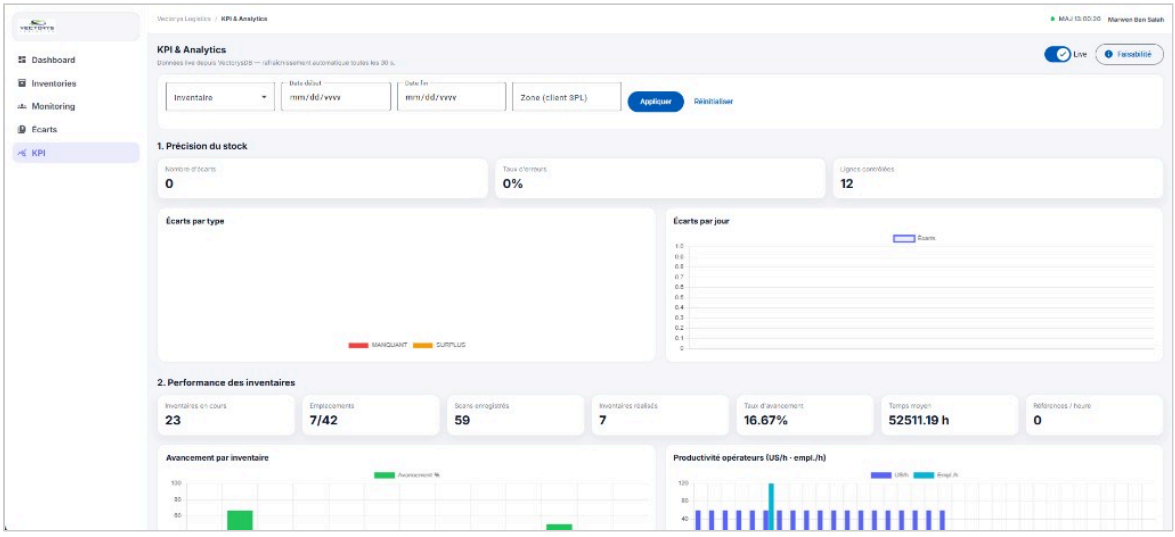



*Figure 3.18 — Tableau de bord KPI*

## 3.4 Vue synthétique de l'API

Le tableau suivant relie les packages du use case global aux principaux endpoints réalisés.

| Domaine | Endpoints principaux | Client |
|---|---|---|
| Authentification | POST /api/auth/login | Web + Mobile |
| Inventaires | GET/POST /api/inventaires ; suivi-temps-reel | Web |
| Import | POST /api/passages/{id}/import-stock | Web |
| Emplacements | GET /emplacements ; GET /mes-emplacements | Web / Mobile |
| Scans | POST /scans ; POST /scans/batch ; fin-comptage | Mobile |
| Écarts | detecter ; branche-a ; GET /ecarts | Web |
| KPI | /kpi/dashboard, precision, performance, qualite, clients, wms | Web |
| Temps réel | Hub /hubs/inventaire | Web |

# Conclusion

Ce chapitre a formalisé la conception de Vectorys Inventaire à partir du use case global : architecture three-tier, modèle de classes métier, puis raffinement fonctionnalité par fonctionnalité (cas d'utilisation détaillés, séquences et classes ciblées). Les emplacements réservés permettront d'intégrer les captures d'écran fournies ultérieurement. Le chapitre 4 présentera la réalisation technique sprint par sprint, en illustrant la mise en œuvre concrète de cette conception.